Amrutam - Telemedicine & Doctor Booking GraphQL API

A robust, enterprise-grade Telemedicine and Doctor Consultation Backend built with **NestJS**, **GraphQL (Apollo Server Code-First)**, **TypeORM**, and **PostgreSQL**.

Table of Contents

- Overview
- Architecture & Tech Stack
- Feature-Based Architecture & Directory Structure
  - Architectural Principles
  - Directory & File Structure
  - Feature Module Anatomy
- Database Schema & Entity Relationship Diagram (ERD)
  - Entity Relationship Diagram
  - Entity Schema Definitions
- Security Architecture & Zero-Trust Model
  - 1. Zero-Trust Access Pipeline
  - 2. Dual-Mode Token Authentication
  - 3. Multi-Factor Authentication (MFA / 2FA) Deep Dive
  - 4. Concurrency Control & Double-Booking Prevention
  - 5. System Audit Logging & Incident Tracking
- Observability & System Monitoring
  - Vercel Serverless Monitoring & Performance Insights
- CI/CD Pipeline & GitHub Actions
  - PR Gatekeeper & Automated Build Validation
  - Vercel Production Deployment Automation
- Development Quality & Pre-Commit Hooks (Husky)
  - Pre-Commit Verification Gates
  - Granular Bypass Environment Variables
- Comprehensive User Flows & Lifecycle Diagrams
  - Lifecycle Sequence Diagram
  - Flow 1: User Onboarding, MFA Enrollment & 2-Step Login Handshake
  - Flow 2: Doctor Profile Setup & Availability Slot Generation
  - Flow 3: Patient Discovery, Concurrency Locking & Consultation Booking
  - Flow 4: Payment Initiation & Processing
  - Flow 5: Consultation Execution & Digital Prescription Issuance
  - Flow 6: Admin Executive Analytics & Incident Audit
- GraphQL Operations Reference
  - Auth & MFA Operations
  - Profile Operations
  - Doctor Operations
  - Availability Slot Operations
  - Consultation Operations
  - Payment Operations
  - Prescription Operations
  - Audit Log Operations

    - Admin Analytics Operations
- API Documentation & Testing with Bruno
    - Bruno Collection Architecture
    - Environment Configuration & Variables
    - Module Directory Breakdown
    - Automated API Testing with Bruno CLI
- Environment Configuration
- Getting Started

Overview

Amrutam API provides an end-to-end telemedicine backend platform connecting patients with verified medical specialists. Key capabilities include:

- **Role-Based Access Control (RBAC)**: Strict role guards for `PATIENT`, `DOCTOR`, and `ADMIN`.
- **UUID Primary & Foreign Keys**: Global UUID v4 identifiers across all domain entities.
- **Doctor Profile & Verification**: Specialized credentials, experience, license numbering, and consultation fees.
- **Availability Slot Engine**: Single slot creation and bulk slot generator with 30-minute intervals and collision prevention.
- **Consultation Lifecycle Management**: Slot reservation (`AVAILABLE` → `BOOKED`), status progression (`PENDING`, `CONFIRMED`, `COMPLETED`, `CANCELLED`, `RESCHEDULED`), notes, and slot release on cancellation.
- **Integrated Payment Flow**: Patient payment initiation returning a structured `CreatePaymentResponse` with `payment_id`, payment processing (`PENDING` → `PAID`), and full refund support.
- **Structured Digital Prescriptions**: Multi-item medication prescribing with dosage, frequency, duration, and patient instructions.
- **Audit Logging & Error Tracking**: Integrated logging in `handleServiceError` capturing failures and audit trails into PostgreSQL with admin queries and user activity inspection.
- **Clean GraphQL Contract**: Action mutations return lightweight confirmation strings (`String`) or structured response objects (e.g. `CreatePaymentResponse`), while queries deliver rich, deeply resolved entity graphs on demand.

Architecture & Tech Stack

- **Framework**: NestJS (v12)
- **API Paradigm**: Code-First GraphQL via @nestjs/graphql & @nestjs/apollo (Apollo Server 4)
- **Database & ORM**: TypeORM with PostgreSQL (Supabase / local PostgreSQL)
- **Authentication**: JWT (JSON Web Tokens) with dual-mode support:
    - `Authorization: Bearer <token>` HTTP header
    - `jwt_access_token` HTTP-only, secure cookies
- **Security & Authorization**:
    - Global `PublicApiGuard` protecting all endpoints by default unless annotated with `@Public()`
    - `RolesGuard` enforcing role restrictions via `@Roles(...)` metadata
    - Parameter decorator `@CurrentUserId()` extracting authenticated subject UUIDs
    - `helmet`, `cors`, and `bcrypt` password hashing
- **Validation**: `class-validator` & `class-transformer` with global validation pipes
- **Runtime**: Bun & Node.js (v20+)

Feature-Based Architecture & Directory Structure

The Amrutam API is designed according to **Feature-Driven Development (FDD)** and **Vertical Slice Architecture** principles. Rather than organizing code by horizontal technical layers (all controllers in one directory, all services in another), each clinical domain is packaged as a high-cohesion, low-coupling feature module.

Architectural Principles

1. **Domain Feature Slicing (`src/features/`)**:
   - Every business capability (`auth` , `doctor` , `consultation` , `payment` , `prescription` , `availability-slot` , `analytics` , `audit-log` , `profile`) is an autonomous domain module containing its own module registration, GraphQL resolvers, business services, input DTOs, response entities, and unit tests.
   - Cross-domain interactions occur cleanly through service dependency injection or repository references without tight coupling.
2. **Strict Separation of Database Schemas vs GraphQL Surface**:
   - **Database Schemas (`src/schemas/`)**: Define the physical relational model in PostgreSQL via TypeORM decorators (`@Entity()` , `@Column()` , `@OneToMany()` , `@JoinColumn()`).
   - **GraphQL Input DTOs (`src/features/*/dto/`)**: Define and enforce incoming mutation and query argument validation using `class-validator` and `class-transformer` .
   - **GraphQL Object Entities (`src/features/*/entities/`)**: Define the outgoing GraphQL response shapes via `@ObjectType()` and `@Field()` , ensuring that database internals (e.g. hashed passwords, internal tokens) are never unintentionally leaked to clients.
3. **Core Infrastructure vs Common Primitives**:
   - `src/core/` : Central infrastructure engines (TypeORM database bootstrap, Redis caching client, Apollo GraphQL driver setup, Helmet/CORS security, Cloudinary media CDN, and request context middleware).
   - `src/common/` : Reusable domain primitives including guards (`JwtAuthGuard` , `RolesGuard`), custom decorators (`@CurrentUser()` , `@Roles()` , `@Public()`), domain enums (`Role` , `ConsultationStatus`), and standard pagination DTOs.
   - `src/utils/` : Deterministic utility functions for cryptographic hashing, TOTP MFA secret derivation, HTTP-only cookie management, and centralized error logging.

Directory & File Structure

```
api/
├── .bruno/                              # Bruno GraphQL API Collection & testing environment
├── .github/
│   └── workflows/
│       └── deploy-vercel.yml            # CI/CD GitHub Actions pipeline
├── .husky/
│   └── pre-commit                       # Pre-commit hook: Lint-staged, Build & Security Audit
├── src/
│   ├── app.module.ts                    # Root NestJS application module wiring all features
│   ├── main.ts                          # Bootstrap entry point (Helmet, CORS, ValidationPipe)
│   │
│   ├── common/                          # Shared domain primitives & cross-cutting contracts
│   │   ├── decorator/                   # Custom decorators (@CurrentUser, @Roles, @Public)
│   │   ├── enum/                        # Global enums (Role, ConsultationStatus, PaymentStatus)
│   │   ├── gaurds/                      # Authorization guards (JwtAuthGuard, RolesGuard)
│   │   └── pagination/                  # Standardized Relay-style / offset pagination DTOs
│   │
│   ├── core/                            # Infrastructure adapters & third-party integrations
│   │   ├── cloudinary/                  # Cloudinary image & document upload service
│   │   ├── context/                     # GraphQL execution context extraction
│   │   ├── database/                    # TypeORM PostgreSQL connection configuration
│   │   ├── graphql/                     # Apollo Server Code-First driver setup
│   │   ├── middleware/                  # HTTP & cookie-parsing middlewares
│   │   ├── redis/                       # Distributed Redis cache & mutex client
│   │   └── security/                    # Helmet headers, rate limiting, and CORS policy
│   │
│   ├── features/                        # Domain-driven vertical feature slices
│   │   ├── analytics/                   # Admin executive KPI aggregation & reporting
│   │   │   ├── dto/                     # Analytics filter inputs
│   │   │   ├── entities/                # GraphQL ObjectTypes for KPI dashboards
│   │   │   ├── analytics.module.ts
│   │   │   ├── analytics.resolver.ts
│   │   │   ├── analytics.service.ts
│   │   │   └── analytics.service.spec.ts
│   │   ├── audit-log/                   # Error tracking, audit log queries & resolution
│   │   ├── auth/                        # Registration, login, JWT issuance, MFA / 2FA TOTP
│   │   ├── availability-slot/           # Doctor slot generation, bulk slots & concurrency locks
│   │   ├── consultation/                # Patient bookings, status transitions & auto-expiry
│   │   ├── doctor/                      # Doctor credentials, verification & search
│   │   ├── health/                      # Uptime probes & database health inspection
│   │   ├── payment/                     # Payment intent creation, processing & refund flows
│   │   ├── prescription/                # Structured digital prescriptions with medications
│   │   └── profile/                     # User & patient demographic profile management
│   │
│   ├── schemas/                         # TypeORM relational entities (PostgreSQL tables)
│   │   ├── audit_log.schema.ts          # audit_logs table
│   │   ├── availability_slot.schema.ts  # availability_slots table
│   │   ├── consultation.schema.ts       # consultations table
│   │   ├── doctor.schema.ts             # doctors table
│   │   ├── payment.schema.ts            # payments table
│   │   ├── prescription.schema.ts       # prescriptions table
│   │   ├── profile.schema.ts            # profiles table
│   │   ├── user.schema.ts               # users table
│   │   └── schemas.ts                   # Central entity barrel export for TypeORM
│   │
│   └── utils/                           # Pure utility functions
│       ├── cookie/                      # HTTP-only cookie serialization
│       ├── crypto/                      # Password hashing & token encryption
│       ├── error/                       # Centralized error handling & audit capture
│       └── mfa/                         # TOTP QR-code generation & secret validation
│
├── dist/                                # Compiled JavaScript build artifacts
├── nest-cli.json                        # NestJS CLI configuration
├── package.json                         # Scripts (lint, build, audit) and dependencies
├── tsconfig.json                        # TypeScript strict compiler configuration
```

```
├── tsconfig.json                # TypeScript strict compiler configuration
└── vercel.json                  # Vercel serverless deployment routing config
```

Feature Module Anatomy

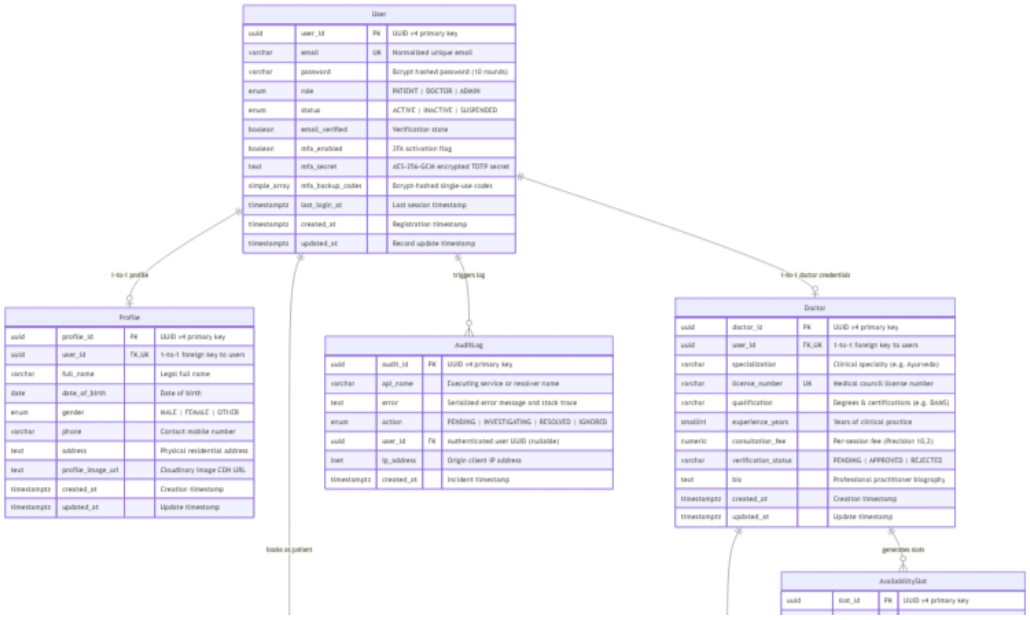
Each domain feature inside `src/features/<feature-name>/` maintains a consistent internal topology:

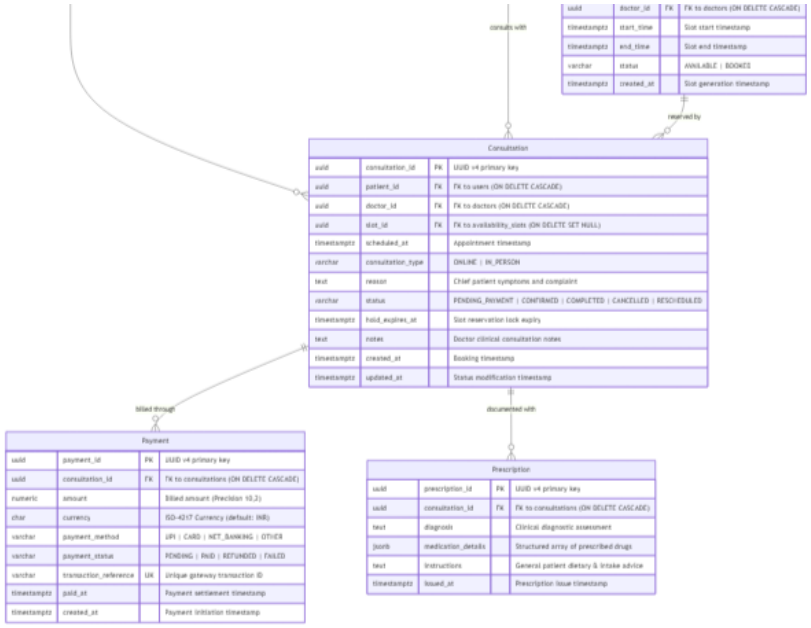
File / Directory	Responsibility	Example (`src/features/consultation`)
`*.module.ts`	Configures NestJS DI container and registers TypeORM repositories.	`consultation.module.ts`
`*.resolver.ts`	Defines GraphQL Query and Mutation entry points, applying `@Roles` and guards.	`consultation.resolver.ts`
`*.service.ts`	Houses business rules, transactional mutations, concurrency locks, and queries.	`consultation.service.ts`
`*.resolver.spec.ts`	Unit tests verifying resolver responses, argument validation, and guards.	`consultation.resolver.spec.ts`
`*.service.spec.ts`	Comprehensive test suite verifying business logic, mocks, and edge cases.	`consultation.service.spec.ts`
`dto/`	Input argument types annotated with `class-validator` for request payload sanitization.	`book-consultation.input.ts`
`entities/`	GraphQL `@ObjectType()` definitions returned to clients across operations.	`consultation.entity.ts`

Database Schema & Entity Relationship Diagram (ERD)

The API is backed by PostgreSQL using TypeORM with global UUID v4 primary keys, strict foreign key constraints, and relational indexes.

Entity Relationship Diagram





Entity Schema Definitions

1. `users` Table

Stores authentication identity credentials, role permissions, account lifecycle status, and multi-factor authentication security state.

- **Constraints**: `email` UNIQUE index.
- **Relationships**: `1:1` with `profiles`, `1:1` with `doctors`, `1:N` with `consultations` (as patient), `1:N` with `audit_logs`.

2. `profiles` Table

Stores personal patient and doctor demographic details.

- **Constraints**: `user_id` UNIQUE foreign key referencing `users(user_id)` with `ON DELETE CASCADE`.

3. `doctors` Table

Maintains medical licensure, clinical qualifications, specialization, verification review status, and consultation fees.

- **Constraints**: `user_id` UNIQUE foreign key, `license_number` UNIQUE.
- **Relationships**: `1:N` with `availability_slots` (`CASCADE`), `1:N` with `consultations` (`CASCADE`).

4. `availability_slots` Table

Stores doctor appointment availability blocks.

- **Constraints**: Composite index on `(doctor_id, status)` for fast lookup during patient discovery.
- **Foreign Key**: `doctor_id` references `doctors(doctor_id)` with `CASCADE` on delete.

5. `consultations` Table

Tracks patient-doctor appointment sessions through their complete state machine: `PENDING_PAYMENT` → `CONFIRMED` → `COMPLETED` (or `CANCELLED` / `RESCHEDULED`).

- **Foreign Keys**: `patient_id` references `users(user_id)`, `doctor_id` references `doctors(doctor_id)`, `slot_id` references `availability_slots(slot_id)` with `ON DELETE SET NULL`.
- **Hold Locks**: `hold_expires_at` tracks temporary slot holds during checkout.

6. `payments` Table

Manages financial transactions, payment gateway references, payment methods, and settlement states (`PENDING` , `PAID` , `REFUNDED`).

- **Constraints**: `transaction_reference` UNIQUE.
- **Foreign Key**: `consultation_id` references `consultations(consultation_id)` with `CASCADE` on delete.

7. `prescriptions` Table

Stores clinical diagnoses and digital medication charts issued by doctors.

- **Column** `medication_details` : PostgreSQL `JSONB` array of structured medication objects:

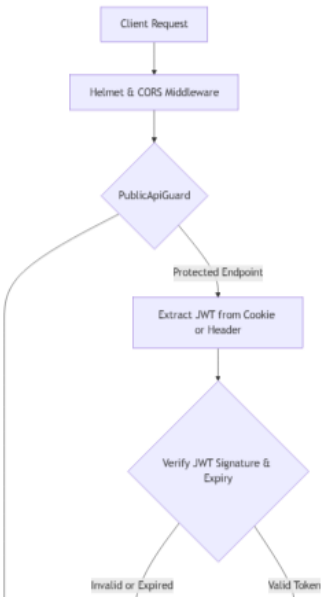
```
[
  {
    "name": "Ashwagandha Tablet",
    "dosage": "500mg",
    "frequency": "Twice daily",
    "duration": "30 days",
    "instructions": "Take after meals with warm milk"
  }
]
```

8. `audit_logs` Table

Maintains security audit events, unhandled application exceptions, API error traces, and administrative resolution workflows (`PENDING` , `INVESTIGATING` , `RESOLVED` , `IGNORED`).

Security Architecture & Zero-Trust Model

The platform enforces a Defense-in-Depth, Zero-Trust architectural design across every layer of the system.





1. Zero-Trust Access Pipeline

1. **Global** `PublicApiGuard` : Every GraphQL query and mutation is protected by default. Only operations decorated with `@Public()` (such as `signup` , `login` , and public doctor queries) bypass the authentication barrier.
2. **Account Status Verification**: Active sessions verify that `user.status === 'ACTIVE'` . Suspended or inactive accounts are rejected immediately before resolving domain data.
3. `RolesGuard` **Enforcement**: Evaluates role requirements specified via `@Roles(UserRole.DOCTOR, UserRole.ADMIN)` . Access is rejected with `ForbiddenException` if the authenticated user's role does not match.
4. **Context Identity Injection**: Resolvers access caller identity via `@CurrentUserId()` and `@CurrentUser()` decorators extracted directly from the verified token payload.

---

2. Dual-Mode Token Authentication

The API provides flexible, secure authentication across web browsers and mobile/native applications:

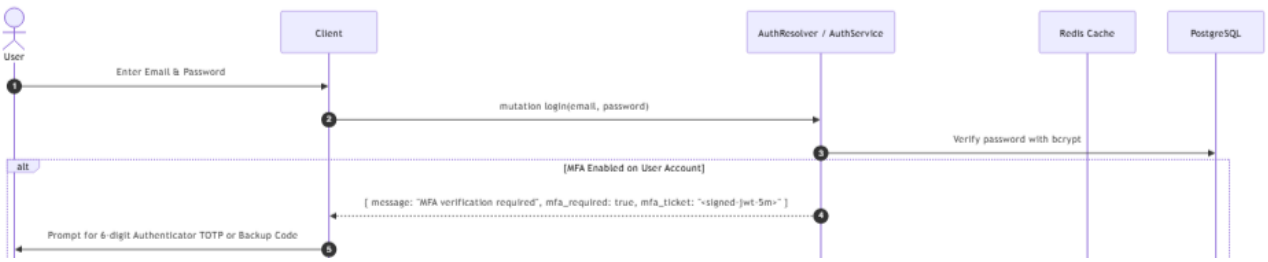
- **Web Clients (Zero-XSS Storage)**: The server automatically issues the `jwt_access_token` as an `HTTP-Only` , `Secure` , `SameSite=Lax` cookie. Web application JavaScript cannot read the token, mitigating Cross-Site Scripting (XSS) exfiltration attacks.
- **Mobile & Third-Party APIs**: Clients can supply the JWT via standard HTTP headers:

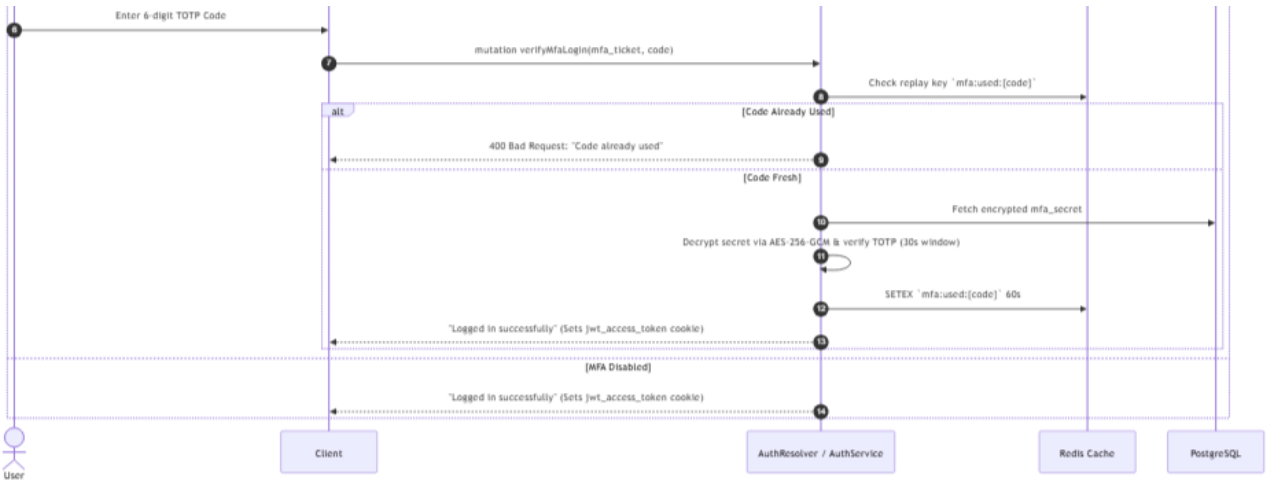
```
Authorization: Bearer <jwt_access_token>
```

---

3. Multi-Factor Authentication (MFA / 2FA) Deep Dive

Amrutam implements a high-security Two-Factor Authentication system based on Time-based One-Time Passwords (TOTP, RFC 6238) and Google Authenticator:





Key MFA Security Features:

1. **2-Step Login Ticket Handshake**:
   - When an MFA-enabled user logs in with email and password, the API does **not** grant access tokens.
   - Instead, it signs a temporary, short-lived `mfa_ticket` (5-minute TTL, `purpose: 'MFA_PENDING'`).
   - The user must call `verifyMfaLogin(input: { mfa_ticket, code })` to exchange the ticket for full access credentials.

2. **AES-256-GCM Authenticated Encryption at Rest**:
   - TOTP secrets are **never stored in plaintext**.
   - Before writing to PostgreSQL, secrets are encrypted using `aes-256-gcm` with random 16-byte initialization vectors (`IV`), authentication tags, and key derivation using `scryptSync` from `JWT_SECRET`.
3. **Redis Replay Attack Prevention**:
   - Each successfully validated 6-digit TOTP code is cached in Redis under `mfa:used:${userId}:${code}` for 60 seconds.
   - Attempting to replay the same TOTP within its 30-second window is rejected with a `400 Bad Request`.
4. **Bcrypt-Hashed Emergency Backup Recovery Codes**:
   - Upon enabling MFA, users receive 8 single-use emergency backup recovery codes (e.g. `a1b2c3d4`).
   - Backup codes are stored in PostgreSQL as `bcrypt` hashes (10 salt rounds).
   - Once a backup code is redeemed during `verifyMfaLogin`, it is atomically deleted from the user's account to prevent reuse.

---

4. Concurrency Control & Double-Booking Prevention

Telemedicine appointment booking requires strict concurrency guarantees to prevent two patients from reserving the same doctor slot simultaneously.

1. **Transactional Slot Status Transitions**:
   - Slots transition atomically: `AVAILABLE` → `BOOKED`.
   - When `bookConsultation` is called, the slot is locked inside a transactional unit of work. If the slot is already `BOOKED`, the transaction immediately aborts with `ConflictException('This slot has already been booked')`.
2. **Hold Reservation Expiry (`hold_expires_at`)**:
   - Initial bookings start in `PENDING_PAYMENT` with a configurable hold timestamp (e.g. 15 minutes).
   - If payment is not completed before hold expiration, scheduled jobs or booking queries release the slot back to `AVAILABLE`.
3. **Atomic Reschedule & Cancellation**:
   - Cancelling a consultation automatically marks the slot back to `AVAILABLE`.
   - Rescheduling atomically frees the old slot and reserves the new slot within a single transaction.

5. System Audit Logging & Incident Tracking

The API integrates real-time error auditing directly into `handleServiceError`:

- **Automated Capture**: Any unhandled internal server error automatically logs the error message, execution stack, calling service name, user ID, and client IP address into the `audit_logs` table.
- **Incident Lifecycle**: Admins can query audit logs by API name, user, or status, and update the investigation lifecycle (`PENDING` → `INVESTIGATING` → `RESOLVED` / `IGNORED`).

Observability & System Monitoring

Amrutam API integrates real-time execution insights and telemetry across serverless production infrastructure:

Vercel Serverless Monitoring & Performance Insights

When deployed to Vercel, the NestJS application runs as a high-performance serverless function, monitored through native Vercel telemetry:

- **Function Invocations & Cold-Start Analysis**: Tracks execution durations, cold-start latency percentiles (p50, p90, p99), and memory usage against configured thresholds.
- **Real-Time Edge Logs**: Live streaming of application stdout/stderr logs with full search and severity filtering (`ERROR`, `WARN`, `INFO`).
- **Speed Insights & Web Vitals**: Analyzes end-to-end response times and TTFB (Time to First Byte) for GraphQL requests across global edge regions.
- **Alert Policies**: Configured alerts notify engineering via Slack, Discord, or email when error rate thresholds or latency spikes exceed acceptable SLOs.

CI/CD Pipeline & GitHub Actions

The repository employs a fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline managed via GitHub Actions (.github/workflows/deploy-vercel.yml).

PR Gatekeeper & Automated Build Validation

Every Pull Request targeting `main` that modifies `api/**` triggers the `build` job:

1. **Environment Setup**: Provisions Ubuntu runner with **Node.js 24** and **Bun v2**.
2. **Deterministic Installs**: Runs `bun install --frozen-lockfile` to ensure exact dependency lockfile parity.
3. **Compilation Check**: Executes `bun run build` to validate TypeScript strict typechecking, decorators, and NestJS build output.
4. **Merge Protection**: GitHub branch protection rules require this check to pass before PRs can be merged into `main`.

Vercel Production Deployment Automation

```
                                                                cancel-in-progress: false
```

1. **Secret Validation**: Asserts that `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`, and `VERCEL_TOKEN` are present in GitHub Actions secrets.
2. **Environment Pull**: Runs `vercel pull --yes --environment=production` to fetch production environment configuration.
3. **Production Artifact Build**: Compiles the optimized Lean package bundle with

5. **Production Artifact Build**: Compiles the optimized serverless bundle with `vercel build --prod`.
4. `vercel deploy --prebuilt --prod`

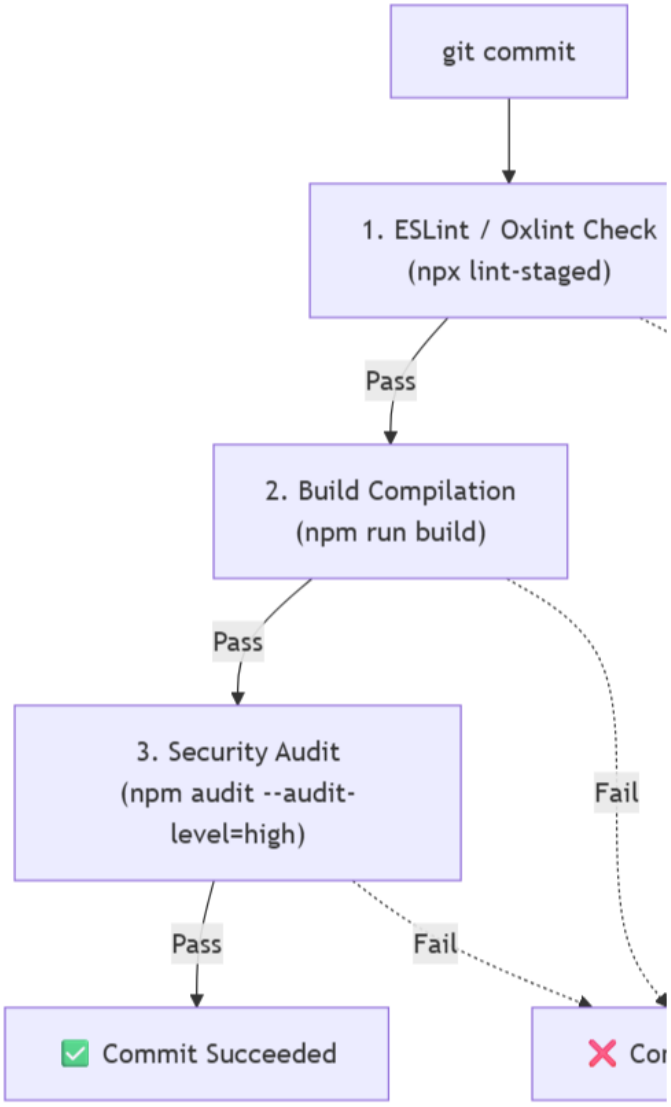
Development Quality & Pre-Commit Hooks (Husky)

The repository integrates **Husky** and **lint-staged** (api/.husky/pre-commit) to enforce strict code hygiene, compilation integrity, and supply chain security prior to any commit:

Pre-Commit Verification Gates

Before git allows any commit to be created, three automated gates execute sequentially:

1. **ESLint / Oxlint Error Check**: Scans all staged TypeScript files (`npx lint-staged` with `oxlint --type-aware`) to catch and intercept syntax errors, unused variables, formatting problems, and floating promises before they enter git history.
2. **NestJS Build Compilation Check**: Runs the complete TypeScript compiler and NestJS build (`npm run build` / `nest build`) to ensure that all modules, services, resolvers, and decorators compile cleanly with zero TypeScript errors.
3. **Dependency Vulnerability Security Audit**: Executes `npm audit --audit-level=high` to inspect all third-party dependencies for known high and critical vulnerabilities, ensuring that vulnerable packages are blocked from entering the codebase.



Granular Bypass Environment Variables

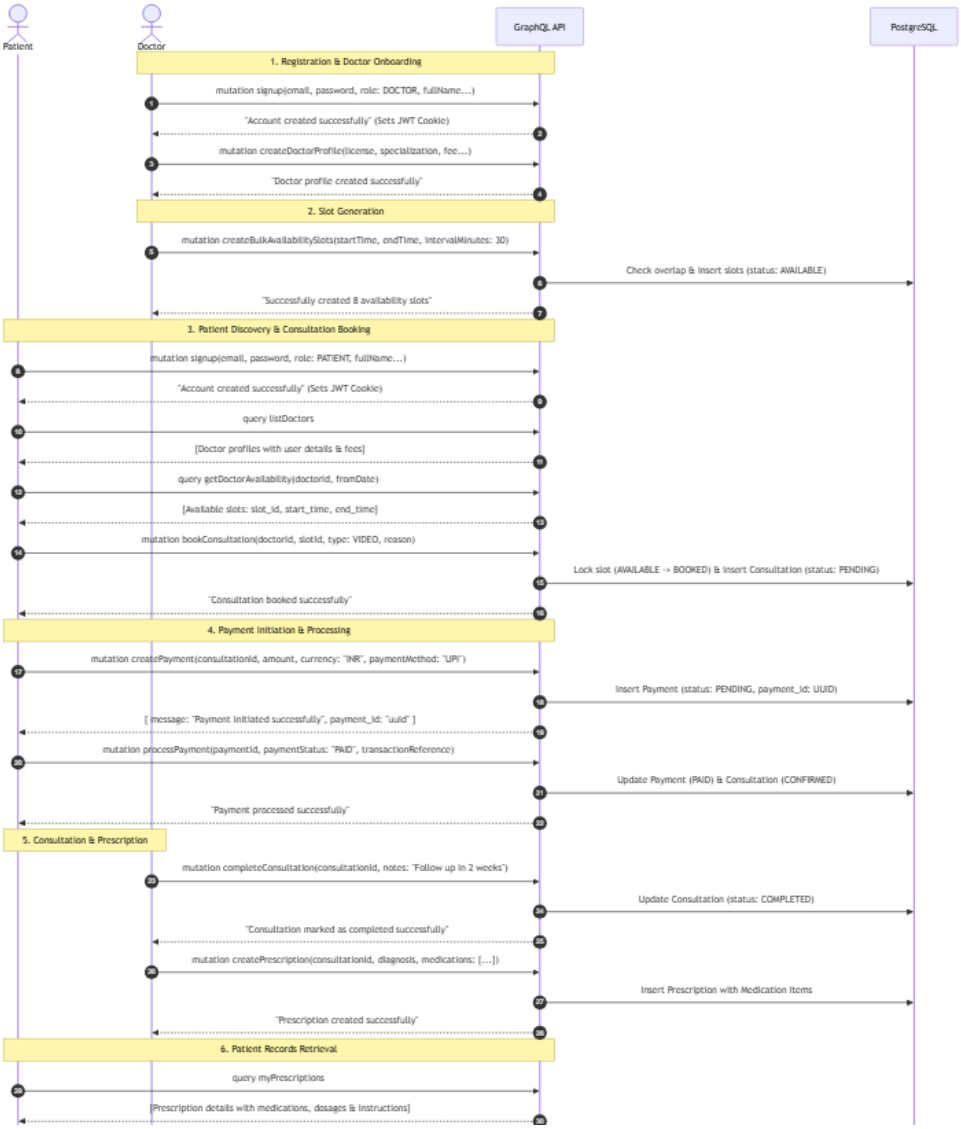
Each verification gate can be individually bypassed when necessary using environment variables:

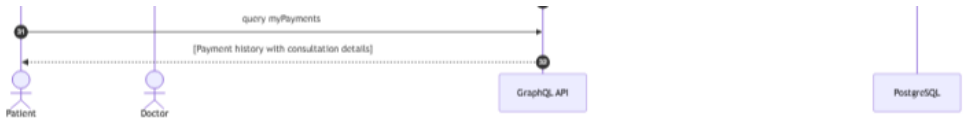
Environment Variable	Target Gate	Description
`SKIP_PRE_COMMIT_LINT=1`	ESLint / Oxlint Check	Skips staged file linting
`SKIP_PRE_COMMIT_BUILD=1`	NestJS Build Compilation	Skips TypeScript project compilation check
`SKIP_PRE_COMMIT_AUDIT=1`	Dependency Security Audit	Skips high/critical vulnerability scanning
`SKIP_PRE_COMMIT=1`	All Validation Gates	Skips all pre-commit hook checks
`HUSKY=0`	Husky Framework	Native Husky bypass (temporarily disables all hooks)

These local pre-commit checks ensure local developer quality while leaving GitHub Actions CI/CD workflows as the immutable final authority on pull requests.

Comprehensive User Flows & Lifecycle Diagrams

Lifecycle Sequence Diagram





Step-by-Step Flow Walkthrough

Flow 1: User Onboarding, MFA Enrollment & 2-Step Login Handshake

A. User Registration (Patient or Doctor)

Both Patients and Doctors create accounts via the `signup` mutation. The API sets a secure HTTP-only `jwt_access_token` cookie and returns a confirmation message.

Doctor Signup Request:

```
mutation DoctorSignup($input: SignupDto!) {
  signup(input: $input)
}
```

Variables:

```
{
  "input": {
    "email": "doctor@amrutam.co.in",
    "password": "Password@123",
    "role": "DOCTOR",
    "full_name": "Dr. Sarah Jenkins",
    "phone": "+919876543210",
    "gender": "FEMALE",
    "date_of_birth": "1988-05-14T00:00:00.000Z"
  }
}
```

Patient Signup Request:

```
mutation PatientSignup($input: SignupDto!) {
  signup(input: $input)
}
```

Variables:

```
{
  "input": {
    "email": "patient@amrutam.co.in",
    "password": "Password@123",
    "role": "PATIENT",
    "full_name": "John Doe",
```

```
    "phone": "+919811122233",
    "gender": "MALE",
    "date_of_birth": "1995-08-20T00:00:00.000Z"
  }
}
```

B. Two-Factor Authentication Setup (Google Authenticator)

Once authenticated, users can enroll in Two-Factor Authentication:

1. **Generate MFA Secret & QR Code:**

```
mutation GenerateMfaSecret {
  generateMfaSecret {
    secret
    qr_code_url
  }
}
```

Response: Returns a Base32 manual entry `secret` and a Data URL PNG `qr_code_url` to scan in Google Authenticator or Authy.

2. **Verify Code & Activate MFA:**
   The user enters the first 6-digit code shown in their app to activate 2FA and receive 8 single-use emergency backup recovery codes:

```
mutation EnableMfa($input: VerifyMfaCodeDto!) {
  enableMfa(input: $input) {
    message
    backup_codes
  }
}
```

Variables:

```
{
  "input": {
    "code": "529148"
  }
}
```

Response:

```
{
  "data": {
    "enableMfa": {
      "message": "MFA has been successfully enabled on your account",
      "backup_codes": [
        "e4a1b2c3", "f5d6e7f8", "a9b0c1d2", "3e4f5a6b",
        "7c8d9e0f", "1a2b3c4d", "5e6f7a8b", "9c0d1e2f"
      ]
    }
```

```
  }
}
```

C. 2-Step Login Handshake

When an account with MFA enabled logs in:

Step 1: Submit Credentials

```
mutation Login($input: LoginDto!) {
  login(input: $input) {
    message
    mfa_required
    mfa_ticket
  }
}
```

Response:

```
{
  "data": {
    "login": {
      "message": "MFA verification required",
      "mfa_required": true,
      "mfa_ticket": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    }
  }
}
```

Step 2: Submit TOTP or Backup Code with Ticket

```
mutation VerifyMfaLogin($input: VerifyMfaLoginDto!) {
  verifyMfaLogin(input: $input)
}
```

Variables:

```
{
  "input": {
    "mfa_ticket": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "code": "529148"
  }
}
```

Result: Returns `"Logged in successfully"`, writes the HTTP-only `jwt_access_token` cookie, records `last_login_at`, and prevents token replay via Redis.

Flow 2: Doctor Profile Setup & Availability Slot Generation

Once logged in, a doctor creates their medical profile and generates booking slots for patients.

Create Doctor Profile:

```
mutation CreateDoctorProfile($input: CreateDoctorDto!) {
  createDoctorProfile(input: $input)
```

```
}
```

Variables:

```
{
  "input": {
    "license_number": "MCI-2026-98745",
    "specialization": "Ayurveda & Holistic Medicine",
    "experience_years": 12,
    "consultation_fee": "750.00",
    "bio": "Specializing in chronic care and Panchakarma therapies."
  }
}
```

Bulk Create Availability Slots:

```
mutation BulkCreateSlots($input: BulkCreateSlotsDto!) {
  createBulkAvailabilitySlots(input: $input)
}
```

Variables:

```
{
  "input": {
    "start_time": "2026-11-01T09:00:00.000Z",
    "end_time": "2026-11-01T13:00:00.000Z",
    "interval_minutes": 30
  }
}
```

Response: `"Successfully created 8 availability slots"`

Flow 3: Patient Discovery, Concurrency Locking & Consultation Booking

Patients discover doctors and inspect open slots without requiring authentication (`@Public()`), then book an appointment using their authenticated session.

List Verified Doctors:

```
query ListDoctors {
  listDoctors {
    doctor_id
    specialization
    experience_years
    consultation_fee
    user {
      profile {
        full_name
      }
    }
  }
}
```

View Doctor's Open Slots:

```
query GetDoctorAvailability($doctorId: ID!, $fromDate: DateTime) {
```

```
  getDoctorAvailability(doctorId: $doctorId, fromDate: $fromDate) {
    slot_id
    start_time
    end_time
    status
  }
}
```

Book Consultation (Patient):

```
mutation BookConsultation($input: CreateConsultationDto!) {
  bookConsultation(input: $input)
}
```

Variables:

```
{
  "input": {
    "doctor_id": "c1f7a0b2-3e4d-5c6f-7a8b-9c0d1e2f3a4b",
    "slot_id": "d2e8b1c3-4f5a-6b7c-8d9e-0f1a2b3c4d5e",
    "consultation_type": "VIDEO",
    "reason": "Persistent digestive fatigue and dietary consultation"
  }
}
```

Result: Slot status transitions atomically to `BOOKED`, consultation is recorded as `PENDING_PAYMENT` with a temporary `hold_expires_at` reservation lock.

Flow 4: Payment Initiation & Processing

The patient initiates a payment for the consultation. The mutation returns both a confirmation message and the newly created `payment_id`.

Initiate Payment:

```
mutation CreatePayment($input: CreatePaymentDto!) {
  createPayment(input: $input) {
    message
    payment_id
  }
}
```

Variables:

```
{
  "input": {
    "consultation_id": "e3f9c2d4-5a6b-7c8d-9e0f-1a2b3c4d5e6f",
    "amount": "750.00",
    "currency": "INR",
    "payment_method": "UPI",
    "transaction_reference": "TXN_UPI_9876543210"
  }
}
```

Response:

```
{
  "data": {
    "createPayment": {
      "message": "Payment initiated successfully",
      "payment_id": "f4a0d3e5-6b7c-8d9e-0f1a-2b3c4d5e6f7a"
    }
  }
}
```

Process Payment (Mark as Paid):

```
mutation ProcessPayment($input: ProcessPaymentDto!) {
  processPayment(input: $input)
}
```

Variables:

```
{
  "input": {
    "payment_id": "f4a0d3e5-6b7c-8d9e-0f1a-2b3c4d5e6f7a",
    "payment_status": "PAID",
    "payment_method": "UPI",
    "transaction_reference": "TXN_UPI_9876543210"
  }
}
```

Result: Payment status is updated to `PAID`, `paid_at` timestamp is set, and the linked consultation is confirmed.

Flow 5: Consultation Execution & Digital Prescription Issuance

The doctor completes the consultation session and issues a digital prescription with medication instructions.

Complete Consultation (Doctor):

```
mutation CompleteConsultation($consultationId: ID!, $notes: String) {
  completeConsultation(consultationId: $consultationId, notes: $notes)
}
```

Variables:

```
{
  "consultationId": "e3f9c2d4-5a6b-7c8d-9e0f-1a2b3c4d5e6f",
  "notes": "Patient advised dietary modifications, herbal supplements, and regular hydration."
}
```

Create Digital Prescription (Doctor):

```
mutation CreatePrescription($input: CreatePrescriptionDto!) {
  createPrescription(input: $input)
}
```

Variables:

```
{
  "input": {
    "consultation_id": "e3f9c2d4-5a6b-7c8d-9e0f-1a2b3c4d5e6f",
    "diagnosis": "Agnimandya (Digestive Weakness)",
    "notes": "Take all medications after meals with warm water.",
    "medications": [
      {
        "name": "Triphala Churna",
        "dosage": "5g",
        "frequency": "Twice daily",
        "duration": "14 days",
        "instructions": "Consume with warm water before bedtime"
      },
      {
        "name": "Trikatu Syrup",
        "dosage": "10ml",
        "frequency": "Once daily",
        "duration": "10 days",
        "instructions": "Take 30 minutes after lunch"
      }
    ]
  }
}
```

Patient Retrieves Prescriptions:

```
query MyPrescriptions {
  myPrescriptions {
    prescription_id
    diagnosis
    notes
    medications {
      name
      dosage
      frequency
      duration
      instructions
    }
    created_at
  }
}
```

Flow 6: Admin Executive Analytics & Incident Audit

Administrators monitor business performance KPIs, doctor capacity metrics, revenue breakdowns, and resolve system exceptions.

Admin Query Dashboard Summary:

```graphql
query AdminDashboardSummary($filter: AnalyticsFilterInput) {
  adminDashboardSummary(filter: $filter) {
    total_revenue
    revenue_growth_percentage
    total_consultations
    completed_consultations
    cancellation_rate
    total_patients
    new_patients_in_period
    total_doctors
    pending_doctor_verifications
    slot_utilization_rate
    active_users_last_30_days
  }
}
```

Admin Inspects System Errors:

```graphql
query GetAuditLogs($action: AuditLogAction) {
  getAuditLogs(action: $action, limit: 10, offset: 0) {
    audit_id
    api_name
    error
    action
    user_id
    ip_address
    created_at
  }
}
```

Admin Resolves Incident:

```graphql
mutation UpdateAuditAction($auditId: ID!, $action: AuditLogAction!) {
  updateAuditLogAction(auditId: $auditId, action: $action)
}
```

Variables:

```json
{
  "auditId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
  "action": "RESOLVED"
}
```

Result: `"Audit log action updated successfully"`

GraphQL Operations Reference

All GraphQL queries and mutations are served at `/graphql`.

Auth & MFA Operations

Type	Operation	Access	Description

`Mutation`	`signup(input: SignupDto!): String!`	`@Public()`	Registers user credentials and profile; sets JWT cookie; returns confirmation.
`Mutation`	`login(input: LoginDto!): LoginResponse!`	`@Public()`	Validates credentials. Returns `mfa_required: true` and `mfa_ticket` if 2FA active, or sets session cookie.
`Mutation`	`verifyMfaLogin(input: VerifyMfaLoginDto!): String!`	`@Public()`	Verifies 6-digit TOTP code or backup code with `mfa_ticket` to complete 2FA login.
`Mutation`	`generateMfaSecret: MfaSetupResponse!`	Protected	Generates Base32 TOTP secret and QR code PNG Data URL for Google Authenticator.
`Mutation`	`enableMfa(input: VerifyMfaCodeDto!): MfaEnableResponse!`	Protected	Verifies first TOTP code, enables 2FA, and returns 8 single-use emergency backup recovery codes.
`Mutation`	`disableMfa(input: VerifyMfaCodeDto!): String!`	Protected	Verifies current TOTP code and disables 2FA on the account.
`Mutation`	`logout: String!`	`@Public()`	Clears `jwt_access_token` authentication cookie.
`Query`	`me: UserEntity!`	Protected	Retrieves authenticated user account, status, role, and nested profile.

Profile Operations

Type	Operation	Access	Description
`Query`	`myProfile: ProfileEntity!`	`PATIENT`, `DOCTOR`, `ADMIN`	Retrieves current user personal details, image URL, and contact info.
`Mutation`	`updateProfile(input: UpdateProfileInput!): String!`	`PATIENT`, `DOCTOR`, `ADMIN`	Modifies name, gender, phone, date of birth, address, or profile image.

Doctor Operations

Type	Operation	Access	Description
`Mutation`	`createDoctorProfile(input: CreateDoctorDto!): String!`	`DOCTOR`	Registers medical license, specialization, experience, and fee.
`Mutation`	`updateDoctorProfile(input: UpdateDoctorDto!): String!`	`DOCTOR`	Updates existing doctor profile details.
`Query`	`myDoctorProfile: DoctorEntity!`	`DOCTOR`	Retrieves the profile for the currently authenticated doctor.
`Query`	`getDoctorById(doctorId: ID!): DoctorEntity!`	`@Public()`	Retrieves public doctor details by doctor UUID.

`Query`	`listDoctors: [DoctorEntity!]!`	`@Public()`	Retrieves all verified registered doctors.

Availability Slot Operations

Type	Operation	Access	Description
`Mutation`	`createAvailabilitySlot(input: CreateAvailabilitySlotDto!): String!`	`DOCTOR`	Adds a single availability slot for the doctor.
`Mutation`	`createBulkAvailabilitySlots(input: BulkCreateSlotsDto!): String!`	`DOCTOR`	Generates consecutive slots split by interval minutes.
`Mutation`	`updateAvailabilitySlot(slotId: ID!, input: UpdateAvailabilitySlotDto!): String!`	`DOCTOR`	Updates slot timing or status (`AVAILABLE` , `LOCKED` , etc.).
`Mutation`	`deleteAvailabilitySlot(slotId: ID!): String!`	`DOCTOR`	Deletes an unbooked slot.
`Query`	`getDoctorAvailability(doctorId: ID!, fromDate: DateTime): [AvailabilitySlotEntity!]!`	`@Public()`	Retrieves upcoming available slots for a doctor.
`Query`	`myAvailabilitySlots: [AvailabilitySlotEntity!]!`	`DOCTOR`	Lists all slots owned by the authenticated doctor.
`Query`	`getAvailabilitySlotById(slotId: ID!): AvailabilitySlotEntity!`	Protected	Retrieves a specific slot by UUID.

Consultation Operations

Type	Operation	Access	Description
`Mutation`	`bookConsultation(input: CreateConsultationDto!): String!`	`PATIENT`	Reserves an available slot and creates a consultation record.
`Mutation`	`rescheduleConsultation(input: RescheduleConsultationDto!): String!`	Protected	Swaps consultation to a new slot and releases the previous one.
`Mutation`	`cancelConsultation(consultationId: ID!, reason: String): String!`	Protected	Cancels consultation and restores slot status to `AVAILABLE` .
`Mutation`	`completeConsultation(consultationId: ID!, notes: String): String!`	`DOCTOR`	Marks consultation as `COMPLETED` and saves clinical notes.
`Query`	`getConsultationById(consultationId: ID!): ConsultationEntity!`	Protected	Retrieves detailed consultation data by ID.
`Query`	`myConsultations: [ConsultationEntity!]!`	Protected	Lists all consultations where user is patient or doctor.

Payment Operations

Type	Operation	Access	Description

`Mutation`	`createPayment(input: CreatePaymentDto!): CreatePaymentResponse!`	`PATIENT`	Initiates a payment; returns `message` and `payment_id`.
`Mutation`	`processPayment(input: ProcessPaymentDto!): String!`	`PATIENT`, `DOCTOR`, `ADMIN`	Updates transaction status (e.g. `PAID`, `FAILED`).
`Mutation`	`refundPayment(paymentId: ID!): String!`	`PATIENT`, `DOCTOR`, `ADMIN`	Refunds a completed payment record.
`Query`	`getPaymentById(paymentId: ID!): PaymentEntity!`	`PATIENT`, `DOCTOR`, `ADMIN`	Fetches payment details and linked consultation.
`Query`	`getPaymentsByConsultation(consultationId: ID!): [PaymentEntity!]!`	`PATIENT`, `DOCTOR`, `ADMIN`	Fetches all payment records for a consultation.
`Query`	`myPayments: [PaymentEntity!]!`	`PATIENT`, `DOCTOR`, `ADMIN`	Returns all payment transactions involving the user.

Prescription Operations

Type	Operation	Access	Description
`Mutation`	`createPrescription(input: CreatePrescriptionDto!): String!`	`DOCTOR`	Issues a digital prescription with medication array.
`Mutation`	`updatePrescription(prescriptionId: ID!, input: UpdatePrescriptionDto!): String!`	`DOCTOR`	Modifies prescription diagnosis, notes, or medications.
`Query`	`getPrescriptionById(prescriptionId: ID!): PrescriptionEntity!`	`@Public()`	Retrieves a single prescription by UUID.
`Query`	`getPrescriptionsByConsultation(consultationId: ID!): [PrescriptionEntity!]!`	`DOCTOR`, `PATIENT`	Retrieves prescriptions for a specific consultation.
`Query`	`myPrescriptions: [PrescriptionEntity!]!`	`DOCTOR`, `PATIENT`	Lists all prescriptions relevant to current patient or doctor.

Audit Log Operations

Type	Operation	Access	Description

Type	Operation	Access	Description
Mutation	`recordAuditLog(input: CreateAuditLogDto!): String!`	ADMIN	Manually writes an audit log entry.
Mutation	`updateAuditLogAction(auditId: ID!, action: AuditLogAction!): String!`	ADMIN	Updates log action status (`PENDING` , `PROCESS` , `FIXED`).
Query	`getAuditLogs(action: AuditLogAction, limit: Int, offset: Int): [AuditLogEntity!]!`	ADMIN	Retrieves paginated system audit & error logs.
Query	`getAuditLogById(auditId: ID!): AuditLogEntity!`	ADMIN	Fetches a single audit log by UUID.
Query	`getAuditLogsByApi(apiName: String!, limit: Int, offset: Int): [AuditLogEntity!]!`	ADMIN	Retrieves audit logs filtered by API name.
Query	`getAuditLogsByUser(userId: ID!, limit: Int, offset: Int): [AuditLogEntity!]!`	ADMIN	Retrieves audit logs for a specific user ID.

```
# Example: Query System Audit & Error Logs (Admin)
query GetAuditLogs($action: AuditLogAction, $limit: Int, $offset: Int) {
  getAuditLogs(action: $action, limit: $limit, offset: $offset) {
    audit_id
    api_name
    error
    action
    user_id
    ip_address
    created_at
  }
}
```

Admin Analytics Operations

Type	Operation	Access	Description
Query	`adminDashboardSummary(filter: AnalyticsFilterInput): AdminDashboardSummaryEntity!`	ADMIN	Top-level KPI overview (revenue, growth %, consults, doctors, patients, slot utilization, active users).
Query	`adminRevenueAnalytics(filter: AnalyticsFilterInput): RevenueAnalyticsEntity!`	ADMIN	In-depth settled revenue time-series, payment methods, transaction values, and payment status breakdown.
Query	`adminConsultationAnalytics(filter: AnalyticsFilterInput): ConsultationAnalyticsEntity!`	ADMIN	Appointment volume timeline, completion/cancellation rates, status distribution, and peak hours.
Query	`adminDoctorAnalytics(filter: AnalyticsFilterInput): DoctorAnalyticsEntity!`	ADMIN	Doctor verification queue, specialization breakdown, slot capacity metrics, and top 10 doctor performers.
Query	`adminPatientAnalytics(filter: AnalyticsFilterInput): PatientAnalyticsEntity!`	ADMIN	Patient growth trends, repeat consultation cohorts, retention rate, and gender demographic distribution.
Query	`adminClinicalAndSystemAnalytics(filter:`		Digital prescription rate, top diagnosed

Query	adminClinicalAndSystemAnalytics(filter: AnalyticsFilterInput): ClinicalAndSystemAnalyticsEntity!	ADMIN	Digital prescription rate, top diagnosed conditions, audit error rates, and top failing API services.

```
# Example: Query Executive Dashboard Summary
query AdminDashboardSummary($filter: AnalyticsFilterInput) {
  adminDashboardSummary(filter: $filter) {
    total_revenue
    revenue_growth_percentage
    total_consultations
    completed_consultations
    cancellation_rate
    total_patients
    new_patients_in_period
    total_doctors
    pending_doctor_verifications
    slot_utilization_rate
    active_users_last_30_days
  }
}
```

Environment Configuration

Create or update `.env` in the `api` root directory (see `.example.env` for reference):

```
# Server & Runtime
PORT=8000
NODE_ENV=development
GRAPHQL_PLAYGROUND=true
CORS_ORIGIN=http://localhost:3000

# Database (PostgreSQL / Supabase)
DATABASE_URL=postgresql://<username>:<password>@<host>:5432/<database>
DATABASE_SSL=true
DATABASE_POOL_MAX=10

# Redis Cache & Concurrency Control
REDIS_URL=redis://localhost:6379
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=

# Authentication & JWT
JWT_SECRET=super-secure-jwt-secret-key-min-32-chars
JWT_EXPIRES_IN=7d
JWT_COOKIE_NAME=jwt_access_token
JWT_COOKIE_MAX_AGE=604800000
BCRYPT_SALT_ROUNDS=10

# Multi-Factor Authentication (MFA / 2FA)
MFA_ENABLED=true
```

```
# Cloudinary Storage (Doctor & Patient Profiles)
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

# NestJS Observability (APM & Distributed Tracing)
OBSERVE_SERVICE_ID=amrutam-api
OBSERVE_APP_KEY=your-observe-app-key
OBSERVE_APP_SECRET=your-observe-app-secret
```

Getting Started

Prerequisites

- Bun (recommended) or Node.js (v20+)
- PostgreSQL 14+ instance

1. Install Dependencies

```
bun install
# or
npm install
```

2. Run the Development Server

```
bun run start:dev
# or
npm run start:dev
```

The Apollo GraphQL Sandbox / Playground will be accessible at:
`http://localhost:8000/graphql`

3. Production Build

```
bun run build
bun run start:prod
```

4. Test Redis Queue and Concurrency

With Redis running and the Redis environment variables configured, run:

```
bun run test:redis
# or
npm run test:redis
```

The script checks Redis connectivity, FIFO processing with simulated concurrent workers, and atomic slot locking under concurrent requests. It uses temporary, unique Redis keys and removes them after the test. This is a Redis smoke test; the API does not currently configure a production job-queue library.

5. Deploying NestJS to Vercel

Vercel supports NestJS as a serverless function. This is the primary cloud deployment platform for Amrutam API, automated through GitHub Actions (.github/workflows/deploy-vercel.yml).

1. Create a Vercel project and import this GitHub repository. Set the Vercel project **Root Directory** to `api`.
2. In Vercel project settings, add the required environment variables (`DATABASE_URL`, `JWT_SECRET`, `DATABASE_SSL`, `MFA_ENABLED`, etc.).
3. In the Vercel dashboard, copy the account/team ID and project ID. Create a Vercel access token.
4. Add these GitHub Actions repository secrets under **Settings → Secrets and variables → Actions**: `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`, and `VERCEL_TOKEN`.
5. Pushes and merges to the `main` branch trigger the CI/CD pipeline, building the prebuilt package and deploying to production automatically with zero downtime.

6. Running with Docker & Docker Compose

Using Docker Compose (API + PostgreSQL):

```
docker compose up --build -d
```

The API will be available at `http://localhost:8000/graphql` and PostgreSQL at `localhost:5432`.

Building and Running the Docker Image directly:

```
# Build the production image
docker build -t amrutam-api .

# Run container with environment configuration
docker run -p 8000:8000 --env-file .env amrutam-api
```

API Documentation & Testing with Bruno

Amrutam API provides a complete, production-ready Bruno API collection located in `api/.bruno/`. Bruno is a fast, Git-friendly, open-source API client that stores request definitions as human-readable `.bru` files directly inside the repository.

Bruno Collection Architecture

The collection is organized by domain module, featuring pre-configured GraphQL operations, variables, headers, and environment definitions:

```
api/.bruno/
├── bruno.json                  # Collection configuration
├── environments/               # Environment configurations
│   └── local.bru               # Local development variables
├── auth/                       # Authentication & MFA requests
├── profile/                    # User profile management
├── doctor/                     # Doctor profiles & verified listings
├── availability-slot/          # Doctor slots & availability
├── consultation/               # Booking & consultation lifecycle
├── payment/                    # Payment initiation & settlement
├── prescription/               # Clinical prescriptions & medications
├── audit-log/                  # Security audit logs & resolution
└── admin-analytics/            # Executive dashboard & domain analytics
```

admin analytics)   # Executive dashboard & domain analytics

Environment Configuration & Variables

Select the `local` environment in Bruno (or create a cloud environment). The default variables configured in `api/.bruno/environments/local.bru` are:

Variable	Default Value	Description
`baseUrl`	`http://localhost:8000`	Root API host address
`graphqlUrl`	`http://localhost:8000/graphql`	Apollo GraphQL endpoint

> **Note on Authentication**: The API automatically issues an HTTP-only `jwt_access_token` cookie upon login or MFA verification. Bruno automatically persists and transmits this cookie for subsequent protected requests.

Module Directory Breakdown

1. Authentication & MFA (`.bruno/auth/`)

- `patient-signup.bru`, `doctor-signup.bru`, `admin-signup.bru`: Create accounts across all system roles.
- `login.bru`: Authenticates credentials; returns `mfa_required: true` and `mfa_ticket` if 2FA is active.
- `generate-mfa-secret.bru`: Generates Base32 TOTP secret and QR code PNG Data URL.
- `enable-mfa.bru`: Verifies first TOTP code and returns 8 emergency recovery backup codes.
- `verify-mfa-login.bru`: Submits 6-digit TOTP code or backup code with `mfa_ticket` to complete 2FA login.
- `disable-mfa.bru`: Disables 2FA upon submitting current TOTP code.
- `me.bru`: Fetches authenticated user identity and nested profile.
- `logout.bru`: Clears session cookie.

2. User Profiles (`.bruno/profile/`)

- `my-profile.bru`: Queries current user profile, phone, address, and profile image URL.
- `update-profile.bru`: Updates profile name, gender, phone, date of birth, address, or Cloudinary image.

3. Doctor Management (`.bruno/doctor/`)

- `create-doctor-profile.bru`: Registers medical license, specialization, experience, fee, and bio.
- `update-doctor-profile.bru`: Modifies doctor practice information.
- `my-doctor-profile.bru`: Retrieves currently authenticated doctor profile.
- `get-doctor-by-id.bru`: Public query to inspect doctor details.
- `list-doctors.bru`: Public listing of all approved medical practitioners.

4. Availability Slots (`.bruno/availability-slot/`)

- `create-availability-slot.bru`: Creates a single availability slot.
- `bulk-create-availability-slots.bru`: Automatically generates consecutive 30-minute appointment slots with overlap protection.
- `get-doctor-availability.bru`: Queries upcoming open slots for a specific doctor.
- `my-availability-slots.bru`: Doctor queries their own slots.
- `update-availability-slot.bru` & `delete-availability-slot.bru`: Manage slot timing and cancellation.

5. Consultations (`.bruno/consultation/`)

- `book-consultation.bru` : Reserves an open slot with atomic lock and creates consultation (`PENDING_PAYMENT`).
- `reschedule-consultation.bru` : Atomically releases previous slot and reserves a new one.
- `cancel-consultation.bru` : Cancels consultation and restores slot to `AVAILABLE` .
- `complete-consultation.bru` : Doctor records consultation notes and marks session `COMPLETED` .
- `my-consultations.bru` & `get-consultation-by-id.bru` : Consultations query.

6. Payments (`.bruno/payment/`)

- `create-payment.bru` : Initiates a payment; returns `payment_id` and payment instructions.
- `process-payment.bru` : Simulates payment gateway callback/webhook, marks payment `PAID` , and confirms consultation.
- `refund-payment.bru` : Issues a full refund for a settled transaction.
- `my-payments.bru` : Patient and doctor transaction history.

7. Digital Prescriptions (`.bruno/prescription/`)

- `create-prescription.bru` : Doctor issues a structured digital prescription with multiple medications (name, dosage, frequency, duration, instructions).
- `update-prescription.bru` : Modifies prescription details.
- `my-prescriptions.bru` : Patient views active and past prescriptions.

8. Audit & Security Logs (`.bruno/audit-log/`)

- `get-audit-logs.bru` : Admin queries paginated error and audit logs.
- `update-audit-log-action.bru` : Admin transitions status (`PENDING` → `INVESTIGATING` → `RESOLVED`).
- `record-audit-log.bru` : Manual audit entry recording.

9. Admin Analytics (`.bruno/admin-analytics/`)

- `get-dashboard-summary.bru` : Executive KPI summary (revenue, growth %, consults, doctors, patients, slot utilization, active users).
- `get-revenue-analytics.bru` : In-depth financial time-series, payment methods, transaction values.
- `get-consultation-analytics.bru` : Appointment timeline, completion rate, status distribution, peak hours.
- `get-doctor-analytics.bru` : Verification queue, specialization breakdown, slot capacity, top 10 performers.
- `get-patient-analytics.bru` : Patient growth trends, repeat consultation cohorts, retention rate, gender demographics.
- `get-clinical-and-system-analytics.bru` : Digital prescription rate, top diagnosed conditions, audit error rates, failing API endpoints.

Automated API Testing with Bruno CLI

The entire Bruno collection can be executed headlessly in CI/CD pipelines or locally using the `@usebruno/cli` :

```
# Install Bruno CLI globally
npm install -g @usebruno/cli

# Run all API requests in the collection against local environment
bru run api/.bruno --env local
```